

UDF – Centro Universitário.

Aluno: Willyan Alves Gomes.

RGM: 43697178.

Professora: Karla Roberto Sartin.

Disciplina: Estrutura de Dados.

Curso: Ciências da Computação.

ATIVIDADE AVALIATIVA – ESTRUTURAS DE DADOS

Arrays, Matrizes, Algoritmos de Ordenação e Busca

Parte 1 – Pesquisa: Bubble Sort e Quick Sort

Bubble Sort

1. Como o algoritmo funciona Percorre o array repetidamente comparando cada par de elementos vizinhos. Se estiverem fora de ordem, troca os dois de posição. A cada passada completa, o maior elemento "borbulha" até a posição correta no final do array, por isso o nome. O processo se repete até que uma passada inteira não precise fazer nenhuma troca — sinal de que o array já está ordenado.

2. Lógica de ordenação Comparação local (par a par) + troca condicional, repetida em múltiplas passadas. Não olha o array como um todo, só decide par a par.

3. Complexidade no melhor caso $O(n)$ — ocorre quando o array já está ordenado. Com uma otimização (parar se nenhuma troca ocorrer numa passada), o algoritmo detecta isso na primeira passada e encerra.

4. Complexidade no caso médio $O(n^2)$ — para dados em ordem aleatória, o número de comparações e trocas cresce proporcionalmente ao quadrado do tamanho do array.

5. Complexidade no pior caso $O(n^2)$ — ocorre quando o array está em ordem totalmente inversa (do maior para o menor), exigindo o número máximo de trocas.

6. Vantagens

- Simples de entender e implementar
- Não precisa de memória extra (ordena "in-place")
- Estável (não muda a ordem relativa de elementos iguais)
- Fácil de detectar quando o array já está ordenado

7. Limitações

- Extremamente ineficiente para arrays grandes ($O(n^2)$)
- Faz muitas trocas desnecessárias comparado a algoritmos mais avançados

8. Situações em que seu uso é adequado

- Arrays pequenos (poucas dezenas de elementos)
- Fins didáticos, para ensinar o conceito de ordenação
- Quando o array já está quase ordenado

9. Situações em que seu uso não é recomendado

- Arrays grandes (centenas ou milhares de elementos) — o tempo de execução cresce rápido demais
- Sistemas com restrição de tempo de resposta

Quick Sort

1. Como o algoritmo funciona Escolhe um elemento como **pivô** e reorganiza o array (etapa de particionamento) de forma que todo elemento menor que o pivô fique à sua esquerda, e todo elemento maior fique à sua direita. O pivô já fica na posição final correta. Em seguida, aplica o mesmo processo recursivamente nas duas partes (esquerda e direita), até que cada partição tenha 0 ou 1 elemento.

2. Lógica de ordenação Divisão e conquista: divide o problema em subproblemas menores (partições), resolve cada um recursivamente e combina os resultados naturalmente (não precisa de etapa de "juntar", diferente do Merge Sort).

3. Complexidade no melhor caso $O(n \log n)$ — ocorre quando o pivô escolhido sempre divide o array em duas metades aproximadamente iguais.

4. Complexidade no caso médio $O(n \log n)$ — mesmo com partições não perfeitamente balanceadas, o comportamento médio se mantém próximo do melhor caso.

5. Complexidade no pior caso $O(n^2)$ — ocorre quando o pivô escolhido é sempre o menor ou o maior elemento da partição (por exemplo, ao ordenar um array já ordenado usando sempre o último elemento como pivô), gerando partições muito desbalanceadas (uma com $n-1$ elementos, outra vazia).

6. Vantagens

- Muito rápido na prática para a grande maioria dos casos ($O(n \log n)$)
- Ordena in-place (não precisa de array auxiliar grande)
- Amplamente usado em bibliotecas padrão de várias linguagens

7. Limitações

- Pior caso é tão ruim quanto o Bubble Sort ($O(n^2)$)
- Não é estável por padrão (pode mudar a ordem relativa de elementos iguais)
- Desempenho depende muito da escolha do pivô

8. Situações em que seu uso é adequado

- Arrays grandes, onde performance importa
- Quando não é necessário manter a ordem relativa entre elementos iguais
- Situação geral de propósito, quando não se conhece a distribuição dos dados

9. Situações em que seu uso não é recomendado

- Quando o array já está ordenado ou quase ordenado e o pivô é escolhido de forma ingênua (ex: sempre o último elemento) — pode cair no pior caso
- Quando a estabilidade da ordenação é um requisito

Tabela Comparativa

Característica	Bubble Sort	Quick Sort
Princípio de funcionamento	Comparação e troca de pares vizinhos, repetidamente	Particionamento em torno de um pivô + recursão (divisão e conquista)
Melhor caso	$O(n)$	$O(n \log n)$
Caso médio	$O(n^2)$	$O(n \log n)$
Pior caso	$O(n^2)$	$O(n^2)$
Uso de memória	In-place, $O(1)$ extra	In-place, mas usa pilha de recursão $O(\log n)$ no melhor caso, $O(n)$ no pior
Vantagem principal	Simplicidade	Velocidade na prática
Limitação principal	Ineficiente em arrays grandes	Pior caso ruim se o pivô for mal escolhido
Aplicação recomendada	Arrays pequenos ou didáticos	Arrays grandes, uso geral